Assignment Documentation

NumPy and Pandas

Name: Aleksandr Chernysh

Batch: 2

Date: 28 October 2026

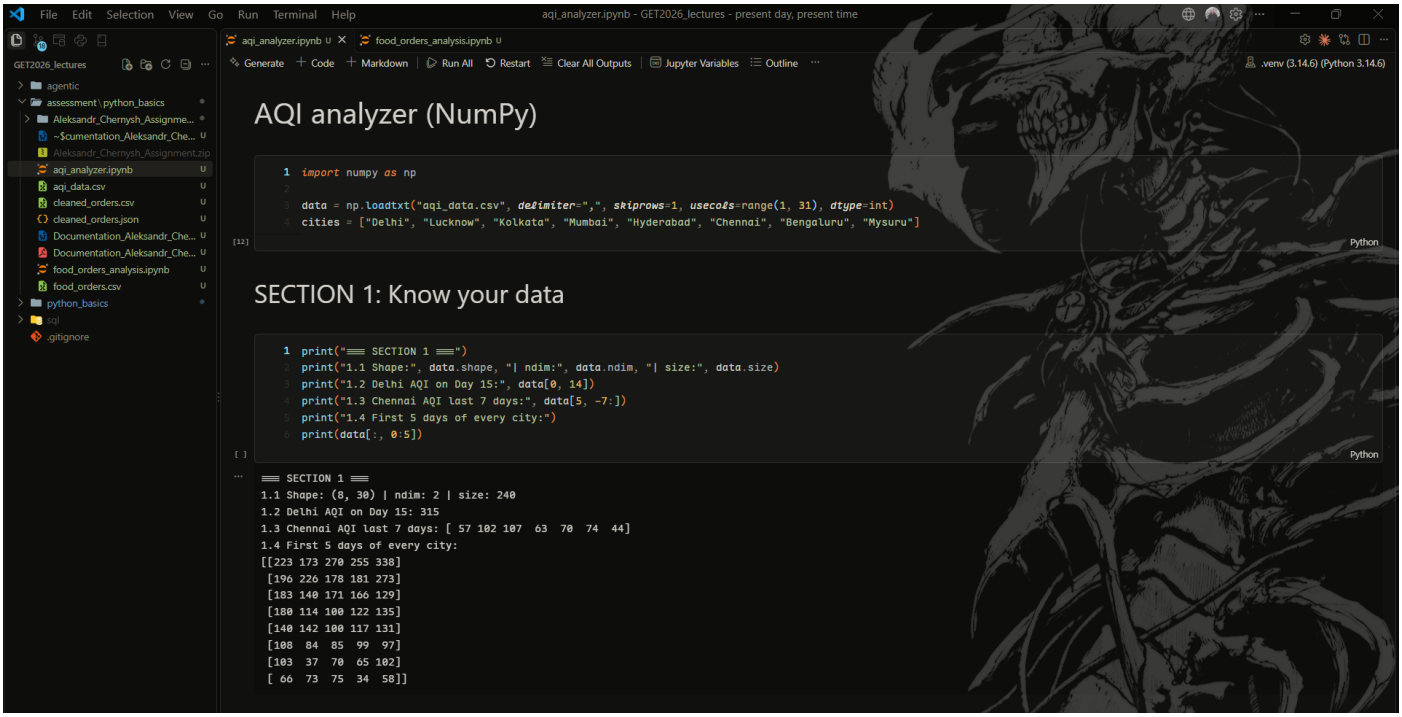
Files submitted:

- numpy_Aleksandr_Chernysh.ipynb
- pandas_Aleksandr_Chernysh.ipynb
- Documentation_Aleksandr_Chernysh.pdf
- Video_Aleksandr_Chernysh.mp4

Part 1: NumPy Report

1.1. Section 1: Know your data

a) Code screenshot and b) Output screenshot



c) Explanation

What I did

Loaded the AQI numbers with the given loadtxt line, used the given list of city names, and checked the array size. Picked out single values and slices.

Why this method

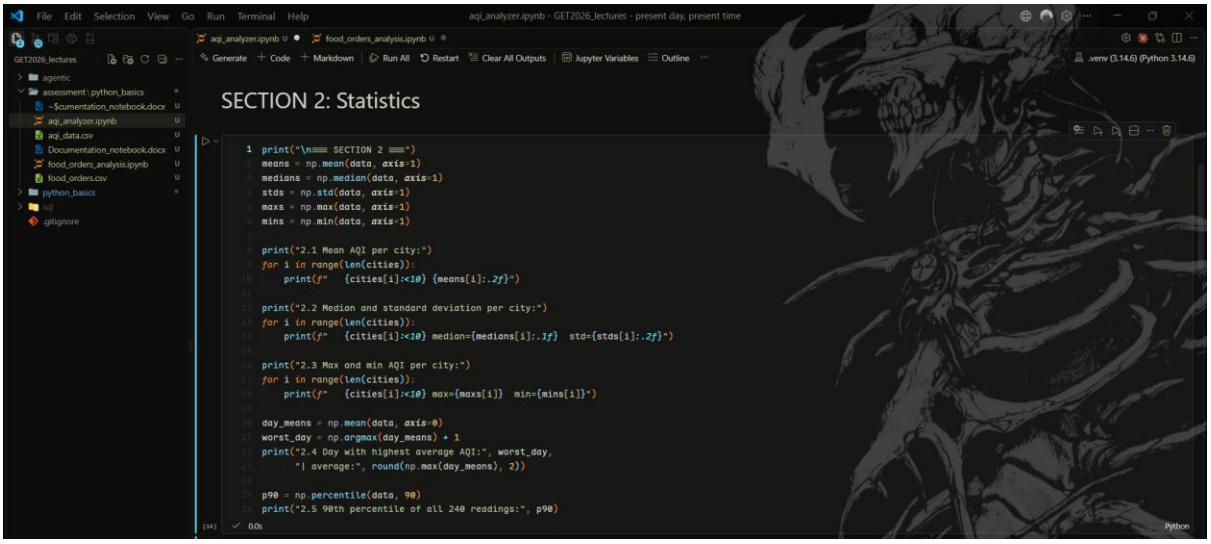
skiprows=1 skips the header and usecols=range(1, 31) takes only the 30 day columns, so the city name column is not read as a number. Day 15 is index 14 because counting starts at 0.

What the result tells me

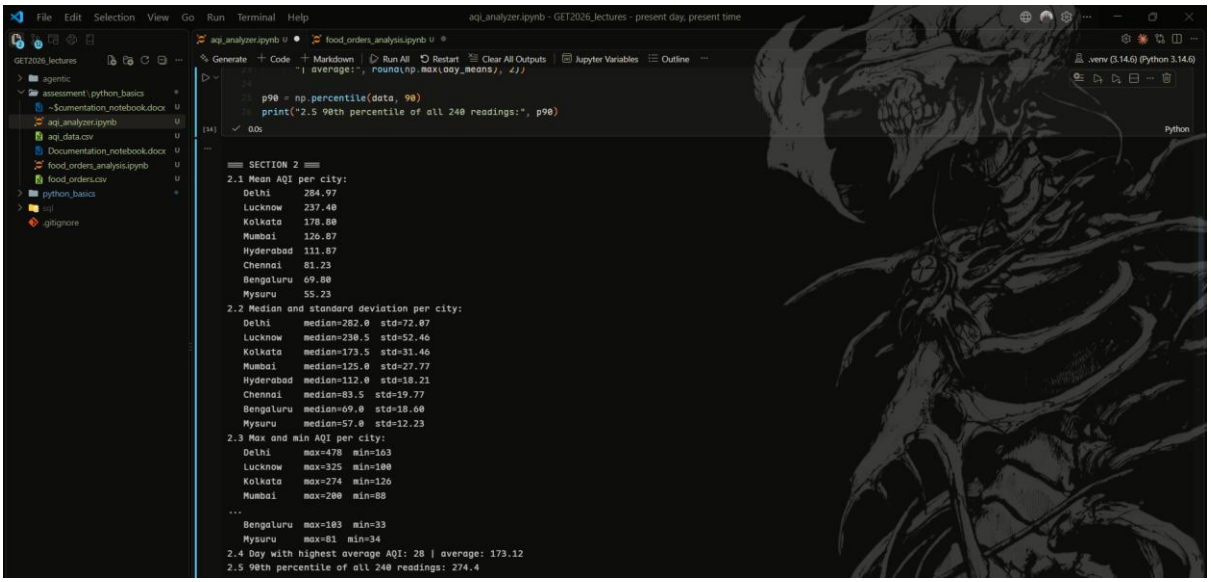
Shape is (8, 30): 8 cities, 30 days. Delhi on Day 15 was 315.

1.2. Section 2: Statistics

a) Code screenshot



b) Output screenshot



c) Explanation

What I did

Found mean, median, std, max and min for each city, the worst day, and the 90th percentile.

Why this method

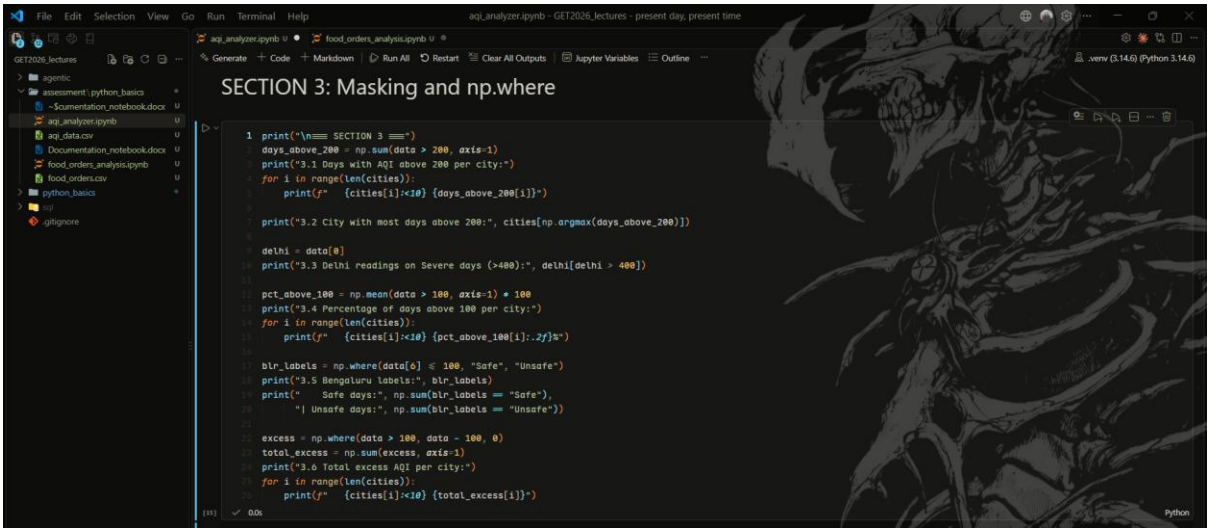
axis=1 goes along a row, so I get one number per city. axis=0 gives one number per day.

What the result tells me

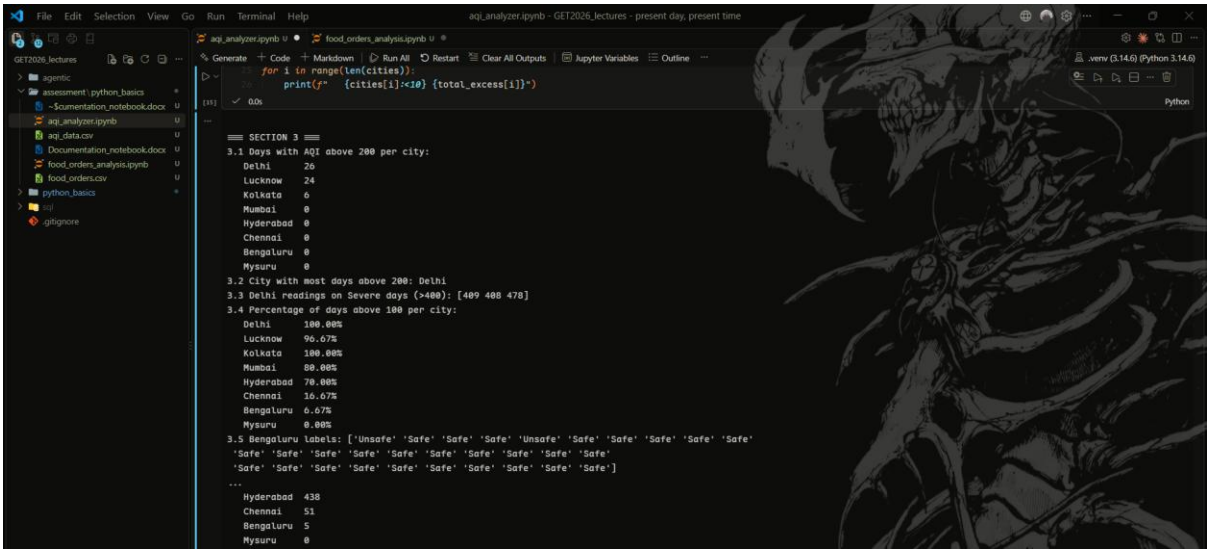
Delhi has the highest mean AQI (284.97). The worst day was Day 28 (average 173.12).

1.3. Section 3: Masking and np.where

a) Code screenshot



b) Output screenshot



c) Explanation

What I did

Counted bad days per city, found Delhi's Severe days, and labelled Bengaluru days as Safe or Unsafe.

Why this method

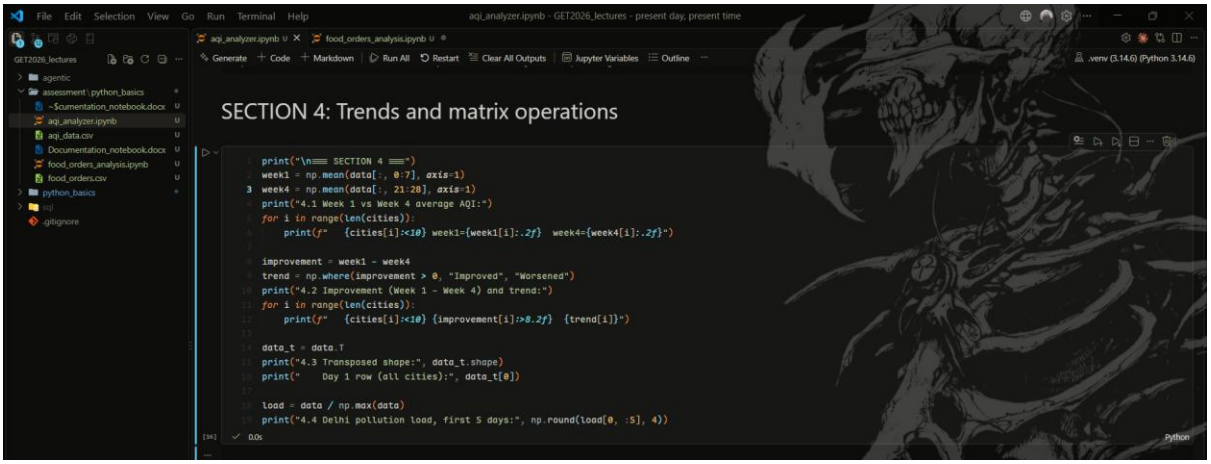
data > 200 gives True/False, and sum() counts the True values. np.where works on the whole array, no loop needed.

What the result tells me

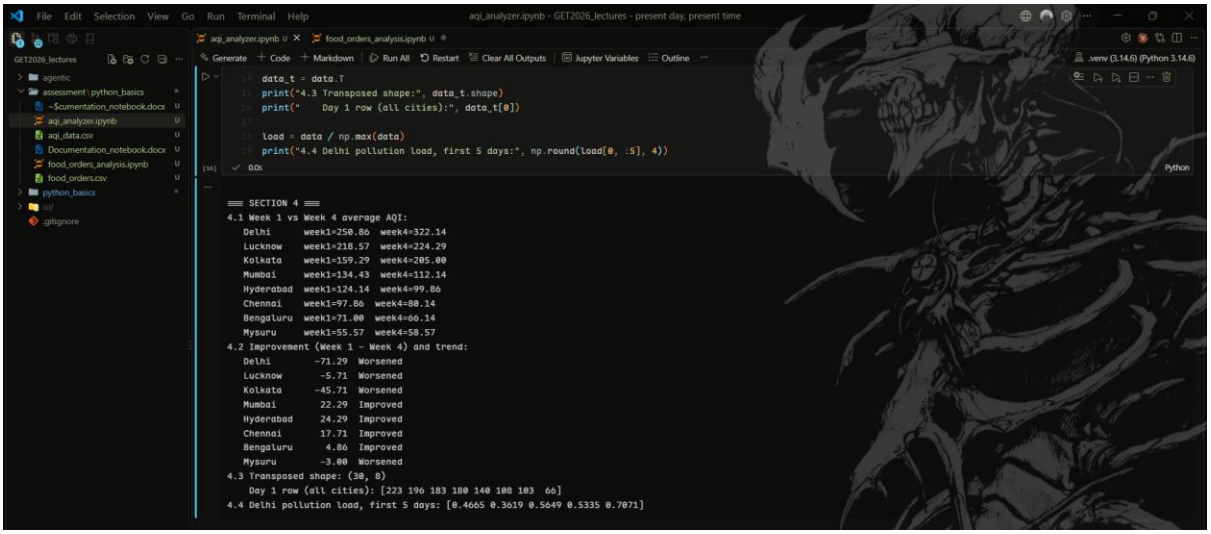
Delhi was above 200 on 26 of 30 days. Bengaluru had 28 Safe days and 2 Unsafe.

1.4. Section 4: Trends and matrix operations

a) Code screenshot



b) Output screenshot



c) Explanation

What I did

Compared Week 1 and 4 for each city, transposed the matrix, and scaled all values from 0 to 1.

Why this method

Week 1 minus Week 4: positive means AQI went down (better). I divided by the overall max (478) so all cities use the same scale.

What the result tells me

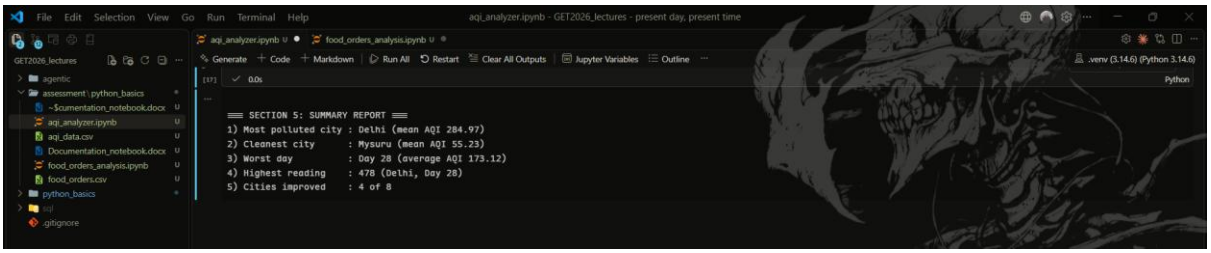
Delhi got worse by 71.29 points. 4 of 8 cities improved.

1.5. Section 5: Report

a) Code screenshot



b) Output screenshot



c) Explanation

What I did

Printed a short summary of the most important results and wrote a conclusion.

Why this method

Reused results from earlier sections. argmax gives a flat position, so // 30 gives the city and % 30 gives the day.

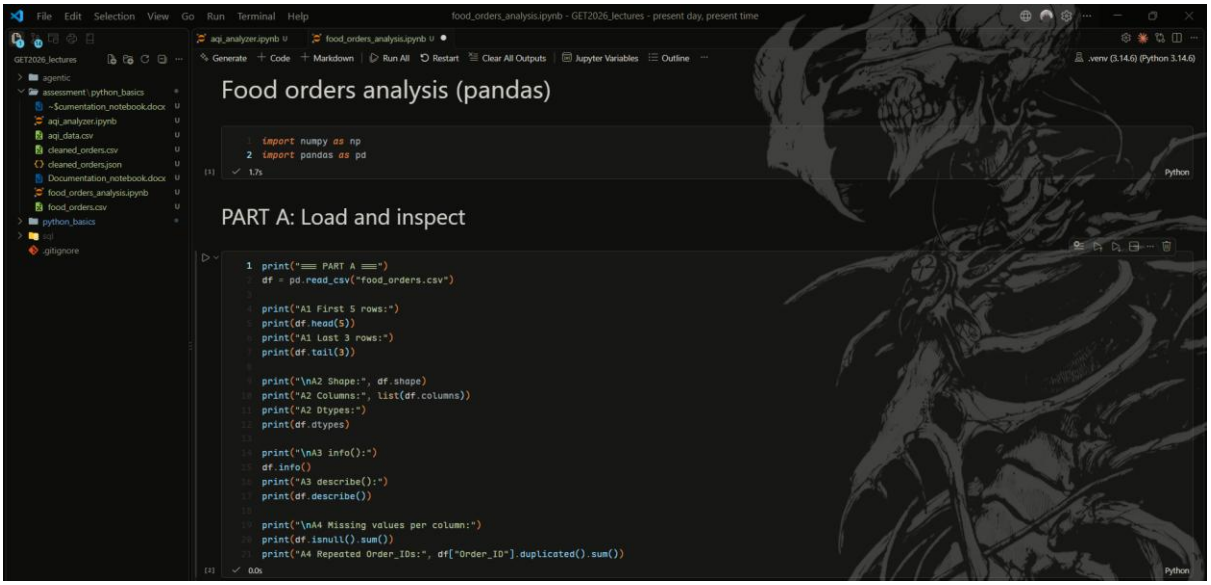
What the result tells me

Most polluted is Delhi (284.97), cleanest is Mysuru (55.23). Highest reading was 478 in Delhi on Day 28.

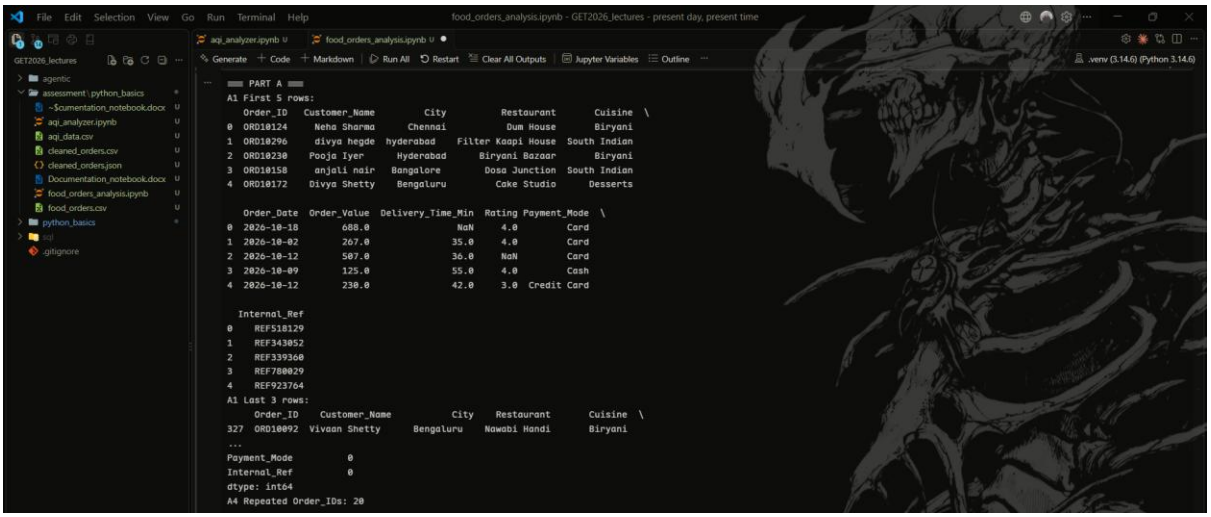
Part 2: Pandas Report

2.1. Part A: Load and inspect

a) Code screenshot



b) Output screenshot



c) Explanation

What I did

Loaded the CSV and looked at the rows, columns, types, missing values and duplicates.

Why this method

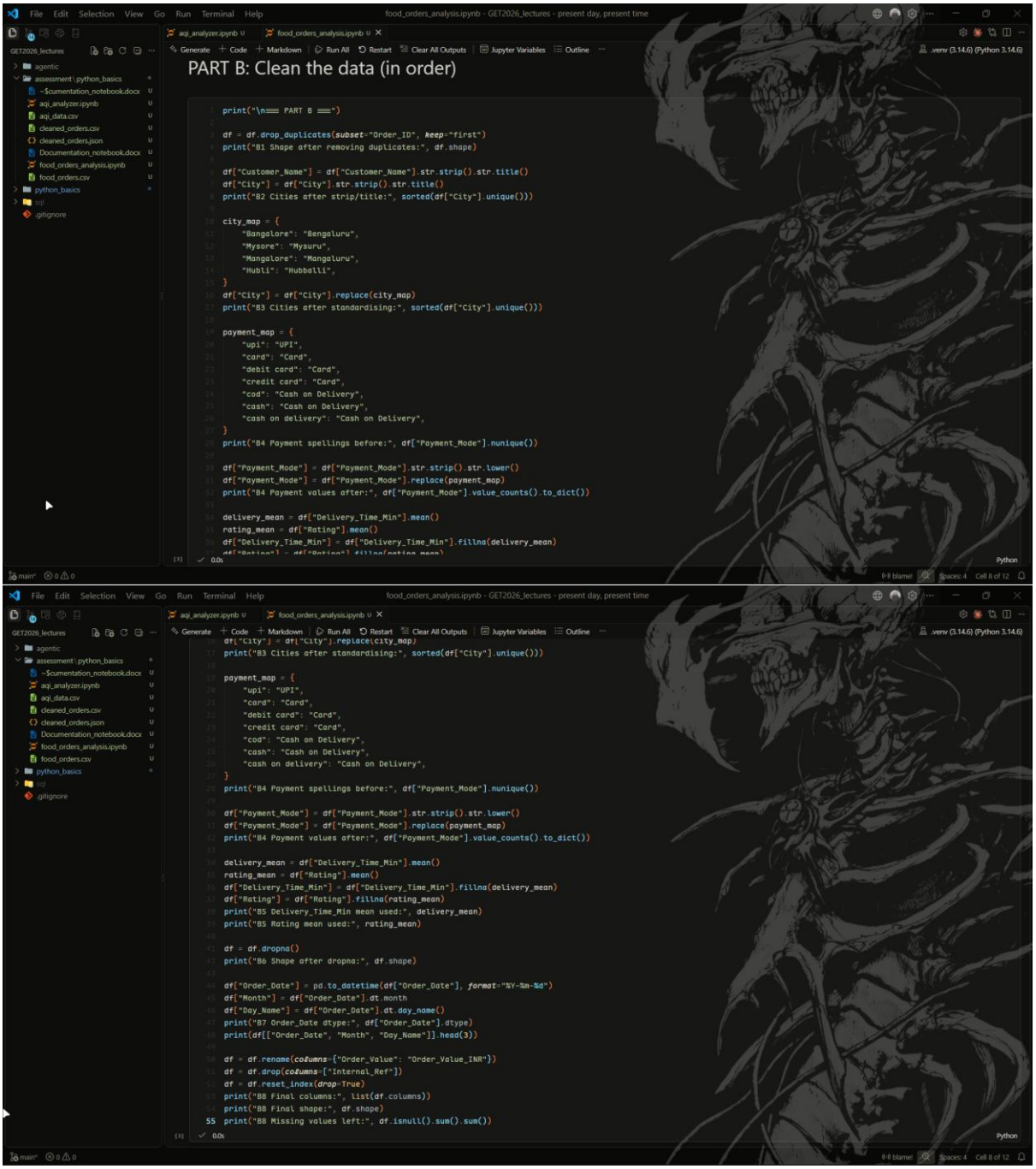
I checked the data first so I know what needs cleaning. I checked duplicates by Order_ID because every order must be unique.

What the result tells me

330 rows, 11 columns. Rating has 34 missing values and there are 20 repeated Order_IDs.

2.2. Part B: Clean the data

a) Code screenshot



```
print("\n== PART B ==")

df = df.drop_duplicates(subset="Order_ID", keep="first")
print("B1 Shape after removing duplicates:", df.shape)

df["Customer_Name"] = df["Customer_Name"].str.strip().str.title()
df["City"] = df["City"].str.strip().str.title()
print("B2 Cities after strip/title:", sorted(df["City"].unique()))

city_map = {
    "Bangalore": "Bengaluru",
    "Mysore": "Mysuru",
    "Mangalore": "Mangaluru",
    "Hubli": "Hubballi",
}
df["City"] = df["City"].replace(city_map)
print("B3 Cities after standardising:", sorted(df["City"].unique()))

payment_map = {
    "upi": "UPI",
    "card": "Card",
    "debit card": "Card",
    "credit card": "Card",
    "cod": "Cash on Delivery",
    "cash": "Cash on Delivery",
    "cash on delivery": "Cash on Delivery",
}
print("B4 Payment spellings before:", df["Payment_Mode"].nunique())

df["Payment_Mode"] = df["Payment_Mode"].str.strip().str.lower()
df["Payment_Mode"] = df["Payment_Mode"].replace(payment_map)
print("B4 Payment values after:", df["Payment_Mode"].value_counts().to_dict())

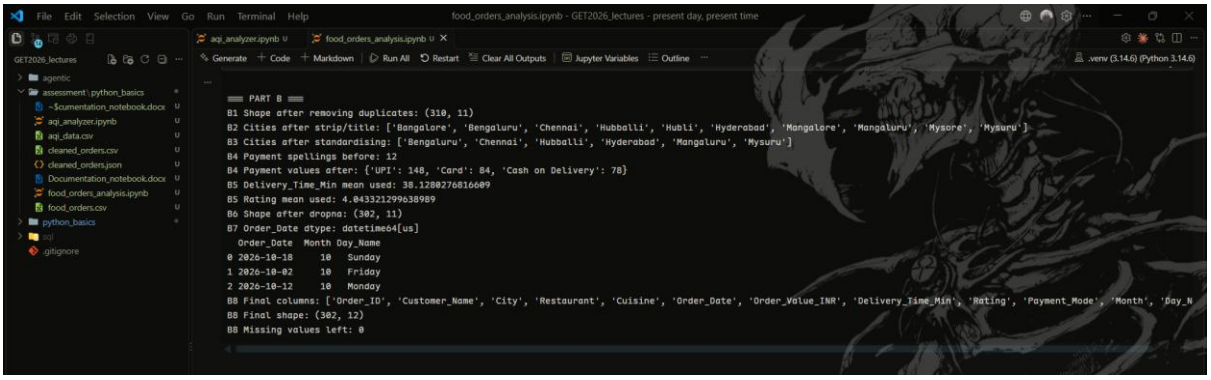
delivery_mean = df["Delivery_Time_Min"].mean()
rating_mean = df["Rating"].mean()
df["Delivery_Time_Min"] = df["Delivery_Time_Min"].fillna(delivery_mean)
df["Rating"] = df["Rating"].fillna(rating_mean)
print("B5 Delivery_Time_Min mean used:", delivery_mean)
print("B5 Rating mean used:", rating_mean)

df = df.dropna()
print("B6 Shape after dropna:", df.shape)

df["Order_Date"] = pd.to_datetime(df["Order_Date"], format="%Y-%m-%d")
df["Month"] = df["Order_Date"].dt.month
df["Day_Name"] = df["Order_Date"].dt.day_name()
print("B7 Order_Date dtype:", df["Order_Date"].dtype)
print(df[["Order_Date", "Month", "Day_Name"]].head(3))

df = df.rename(columns={"Order_Value": "Order_Value_INR"})
df = df.drop(columns=["Internal_Ref"])
df = df.reset_index(drop=True)
print("B8 Final columns:", list(df.columns))
print("B8 Final shape:", df.shape)
print("B8 Missing values left:", df.isnull().sum().sum())
```

b) Output screenshot



```
== PART B ==
B1 Shape after removing duplicates: (310, 11)
B2 Cities after strip/title: ['Bangalore', 'Bengaluru', 'Chennai', 'Hubballi', 'Hubli', 'Hyderabad', 'Mangalore', 'Mangaluru', 'Mysore', 'Mysuru']
B3 Cities after standardising: ['Bengaluru', 'Chennai', 'Hubballi', 'Hyderabad', 'Mangaluru', 'Mysuru']
B4 Payment spellings before: 12
B4 Payment values after: {'UPI': 148, 'Card': 84, 'Cash on Delivery': 78}
B5 Delivery_Time_Min mean used: 38.1280276816609
B5 Rating mean used: 4.043321299638989
B6 Shape after dropna: (302, 11)
B7 Order_Date dtype: datetime64[us]
  Order_Date  Month Day_Name
0 2026-10-18     10   Sunday
1 2026-10-02     10   Friday
2 2026-10-12     10   Monday
B8 Final columns: ['Order_ID', 'Customer_Name', 'City', 'Restaurant', 'Cuisine', 'Order_Date', 'Order_Value_INR', 'Delivery_Time_Min', 'Rating', 'Payment_Mode', 'Month', 'Day_N
B8 Final shape: (302, 12)
B8 Missing values left: 0
```

c) Explanation

What I did

Removed duplicates, fixed spaces and spelling in cities, made payments lowercase and mapped them to 3 values, filled gaps, dropped bad rows, fixed dates.

Why this method

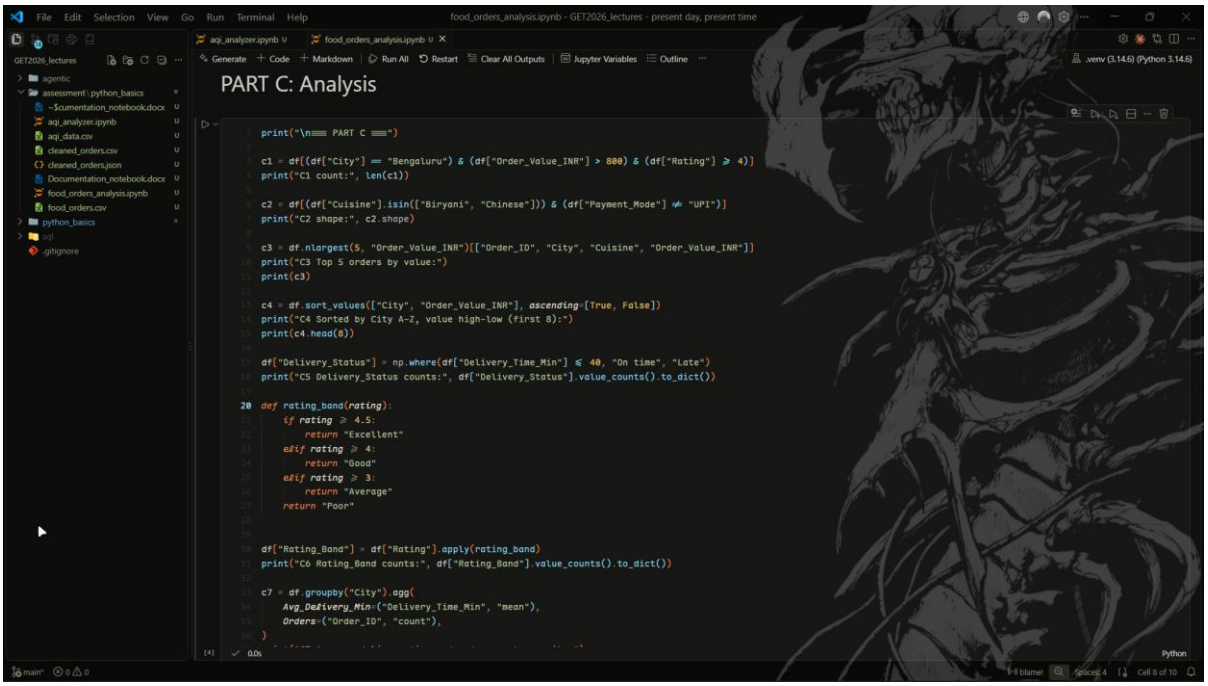
I filled missing values before dropna(), otherwise dropna() would delete rows that only miss a rating. I did not fill Order_Value because I can't guess money.

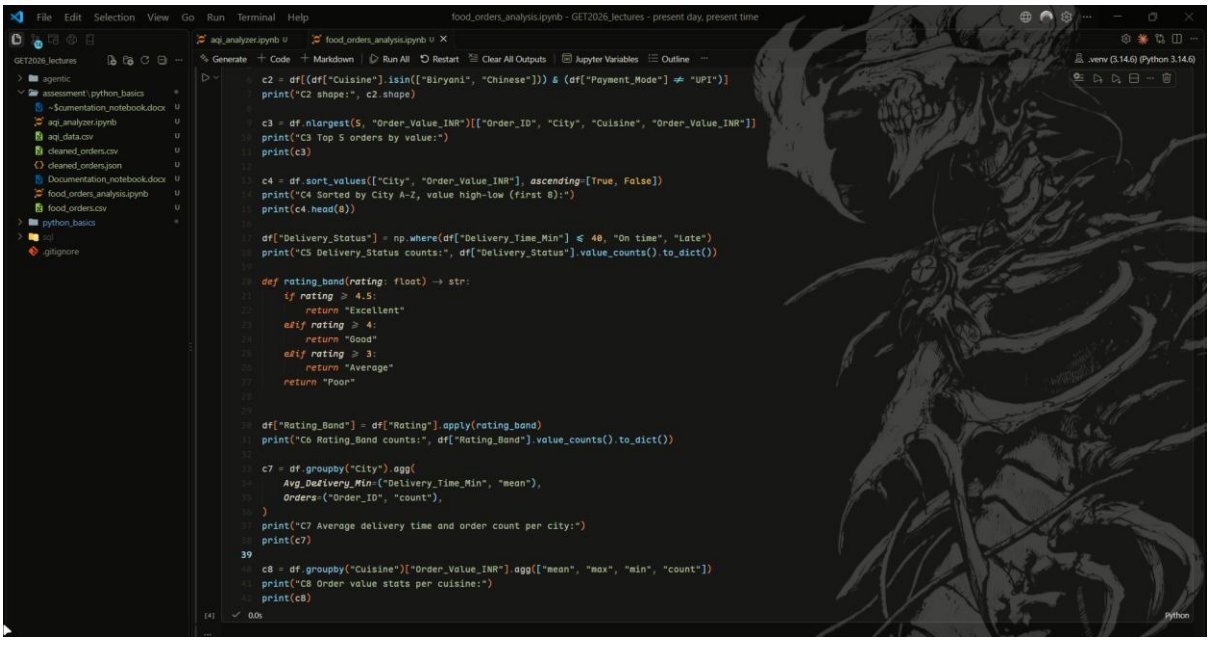
What the result tells me

From 330 rows to 302, with 0 missing values left. Payment had 12 spellings, now 3.

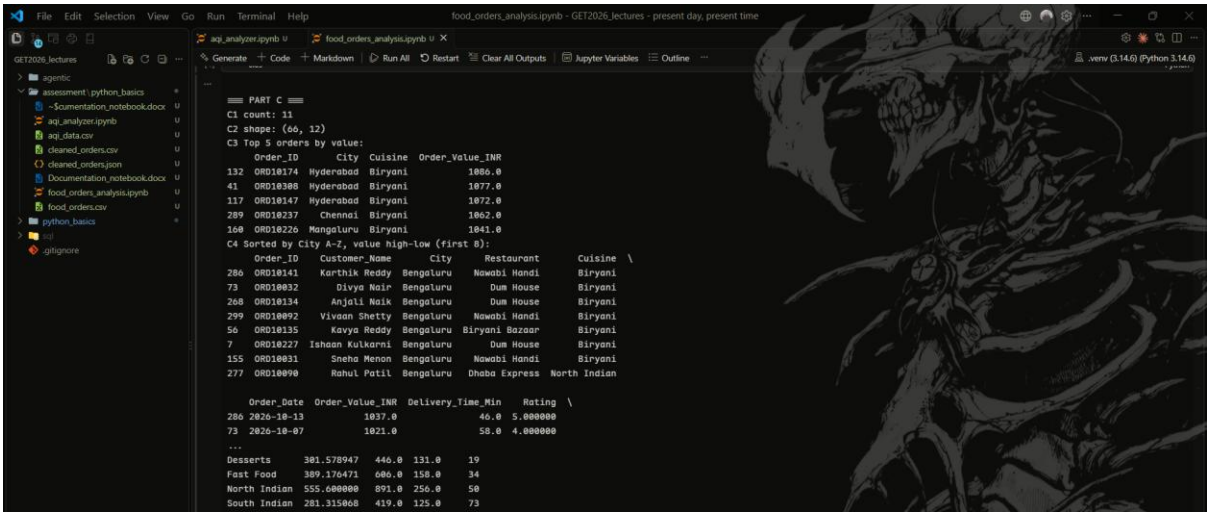
2.3. Part C: Analysis

a) Code screenshot





b) Output screenshot



```
== PART C ==
C1 count: 11
C2 shape: (66, 12)
C3 Top 5 orders by value:
     Order_ID       City  Cuisine  Order_Value_INR
132  ORD10174  Hyderabad  Biryani           1086.0
41   ORD10308  Hyderabad  Biryani           1077.0
117  ORD10147  Hyderabad  Biryani           1072.0
289  ORD10237    Chennai  Biryani           1062.0
160  ORD10226  Mangaluru  Biryani           1041.0
C4 Sorted by City A-Z, value high-low (first 8):
     Order_ID    Customer_Name       City      Restaurant       Cuisine  \
286  ORD10141    Karthik Reddy  Bengaluru    Nawabi Handi      Biryani
73   ORD10032       Divya Nair  Bengaluru       Dum House      Biryani
268  ORD10134      Anjali Naik  Bengaluru       Dum House      Biryani
299  ORD10092    Vivaan Shetty  Bengaluru    Nawabi Handi      Biryani
56   ORD10135       Kavya Reddy  Bengaluru  Biryani Bazaar      Biryani
7    ORD10227  Ishaan Kulkarni  Bengaluru       Dum House      Biryani
155  ORD10031      Sneha Menon  Bengaluru    Nawabi Handi      Biryani
277  ORD10090      Rahul Patil  Bengaluru   Dhaba Express  North Indian

    Order_Date  Order_Value_INR  Delivery_Time_Min    Rating  \
286 2026-10-13          1037.0               46.0  5.000000
73  2026-10-07          1021.0               58.0  4.000000
...
Desserts      301.578947   446.0  131.0     19
Fast Food     389.176471   606.0  158.0     34
North Indian  555.600000   891.0  256.0     50
South Indian  281.315068   419.0  125.0     73
```

c) Explanation

What I did

Filtered orders, found the top 5, sorted, added On time/Late and rating band columns, grouped by city and cuisine.

Why this method

I used & instead of "and" because "and" doesn't work on a whole column. Each condition needs its own brackets.

What the result tells me

116 of 302 orders were Late. Bengaluru is slowest (44.5 min) and Biryani is most expensive (717.15 INR).

2.4. Part D: Export and verify

a) Code screenshot and b) Output screenshot (cleaned_orders.csv and cleaned_orders.json are visible in the folder on the left, plus both printed shapes)



```
PART D: Export and verify

print("\n== PART D ==")
df.to_csv("cleaned_orders.csv", index=False)
df.to_json("cleaned_orders.json", orient="records", indent=4, date_format="iso")
print("D1 Saved cleaned_orders.csv and cleaned_orders.json")

check = pd.read_json("cleaned_orders.json")
print("D2 JSON shape:", check.shape, "| Cleaned DataFrame shape:", df.shape)
print("D2 Match:", check.shape == df.shape)

0.0s

== PART D ==
D1 Saved cleaned_orders.csv and cleaned_orders.json
D2 JSON shape: (302, 14) | Cleaned DataFrame shape: (302, 14)
D2 Match: True
```

c) Explanation

What I did

Saved the clean data as CSV and JSON, then loaded the JSON back to check it.

Why this method

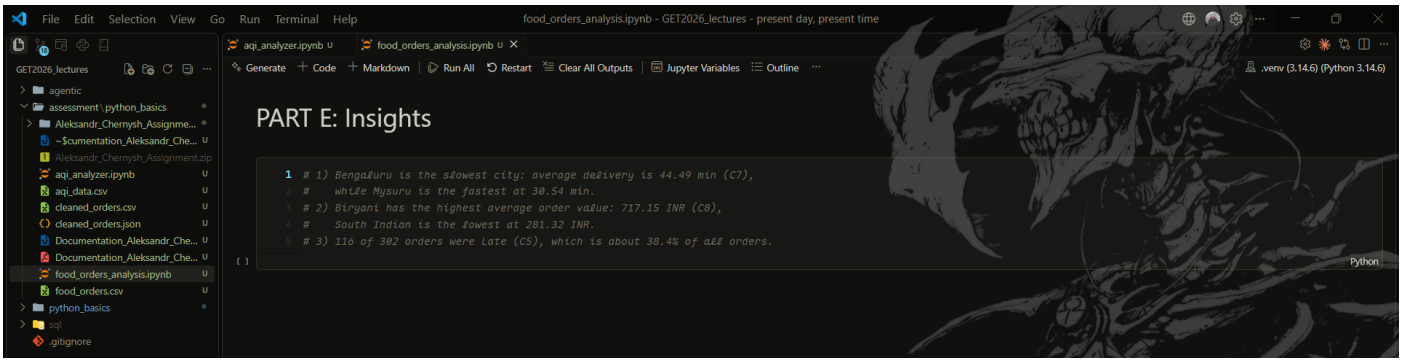
index=False stops an extra index column. Loading the file back proves nothing was lost.

What the result tells me

Both shapes are (302, 14), so the file matches the data.

2.5. Part E: Insights

a) Code screenshot



Declaration

AI tool usage: I used Claude to help me generate the skeleton of this report as well as prettify the print statements in my programs.